ПРАКТИЧЕСКАЯ РАБОТА · МДК.05.03

# Bash, часть 1: отдельные тесты и общий запуск

Выполнила: Миронова Анна Андреевна
Группа: 4-ИСП9-46

Преподаватель: А. Кондыбаева
Дата выполнения: 08.11.2026
Репозиторий: https://github.com/RealLalka/mdk-coursework-mironova-2026

## Цель работы

Проверить корректность работы ИС и научиться обнаруживать воспроизводимые дефекты.

Условие: practice_rose_test_bash_system_part_0.md.

## Выполненная работа

Созданы отдельные test_file.sh, test_lines.sh, test_content.sh, test_rights.sh, test_process.sh, test_port.sh, test_http.sh, test_http_code.sh, test_http_content.sh, test_no_errors.sh. Дополнительно: каталог логов, формат CSV каждой строки, уникальные ID, запись в каталог логов и время ответа. check.sh сохраняет дату и результат каждой проверки. run_all.sh не теряет код ошибки дочернего скрипта.

## Стенд и воспроизведение

Фактическая ОС: Arch Linux в WSL/Linux, ядро 6.6.87.2-microsoft-standard-WSL2. Python 3.14.6, SQLite 3.53.3, Flask 3.1.2; браузер Chromium; настольный Postman 9.31.30 и Newman 6.2.1 для API-прогонов. Условия некоторых заданий называют POCA и Python 3.8: переносимые команды выполнены на текущем Linux; совместимость именно с POCA и Python 3.8 отдельным прогоном не подтверждена. Ограничения окружения отражены как WARN/SKIP, а не как успешные проверки.

Команды установки и запуска приведены в README. Полный автоматический прогон: `python tools/verify.py --workspace <новый-каталог-результатов> --newman <путь-к-newman>`. Он создаёт изолированные копии, запускает локальные сервисы и отрицательные сценарии, затем останавливает собственные процессы. Ненулевой код ожидается только в явно помеченных отрицательных опытах.

## Анализ сложности

Файловые проверки test имеют O(1) метаданных. Чтение CSV O(B), поиск дубликатов ID с sort O(n log n). HTTP-проверки ограничены max-time/timeout, чтобы недоступный сервер не зависал на неопределённый срок.

### Дубликат задания и самостоятельная работа

Файлы practice_rose_test_bash_system.md и practice_rose_test_bash_system_part_0.md побайтно совпадают; им соответствует одно решение. Дополнительные задания выполнены: logs существует и доступен для записи; число строк равно 3, включая заголовок; каждая строка содержит 3 поля; ID

уникальны. Условие отсутствия error проверяется только после успешного получения HTTP-тела. Это исправляет ложный PASS из отрицания grep в исходном примере.

## Скриншоты и примеры выполнения




Рисунок 1. bash_basics_healthy. Фактический запуск; для консольных работ показан оформленный протокол вывода, для веб-приложений и Postman — интерфейс программы.

**Протокол выполнения · bash_real_stop**

Миронова Анна Андреевна · 4-ИСП9-46 · Фактический вывод программы

```
2026-11-08 19:37:25 PASS: Заголовок CSV
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 PASS: Каждая строка содержит 3 поля
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 FAIL: HTTP доступен
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 FAIL: HTTP-код 200
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 FAIL: HTTP содержит Ivan
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 FAIL: Успешный ответ без error
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 FAIL: TCP-порт открыт
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 FAIL: Ответ быстрее 1 секунды
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:37:25 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:37:25 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0
```

Рисунок 2. bash_real_stop. Фактический запуск; для консольных работ показан оформленный протокол вывода, для веб-приложений и Postman — интерфейс программы.

# Ответы на контрольные вопросы

### 1. Что такое код возврата и как его проверить?

Число, возвращаемое процессом оболочке: 0 — успех, ненулевое значение — ошибка. Сразу после команды выполните echo “$?”. В наших агрегаторах наличие хотя бы одного FAIL даёт exit 1.

### 2. Зачем shift в check?

Первый аргумент — название проверки. shift удаляет его из позиционных параметров; “$@” становится выполняемой командой с её аргументами.

### 3. Почему >/dev/null 2>&1 скрывает весь вывод?

Сначала stdout перенаправляется в /dev/null, затем stderr связывается с уже перенаправленным stdout. В реализации отчёта диагностика сохраняется в служебных файлах вместо потери.

### 4. Файл существует и сервер отдаёт файл — одно и то же?

Нет. Файл может быть на диске при остановленном сервере, неправильном URL, запрете доступа или неверном корне публикации. Проверки диска и HTTP независимы.

### 5. Для чего timeout?

Ограничивает ожидание соединения. Без него недоступный адрес может задержать весь прогон на системный TCP-таймаут.

### 6. Что делает ! перед командой?

Инвертирует код успеха: 0 становится 1, ненулевой код становится 0. Нельзя отрицать поиск error в пустом ответе, не проверив успешность curl: иначе недоступный сервер ошибочно даёт PASS.

### 7. Почему после остановки сервера проходят некоторые проверки?

Проверки локального файла, его структуры и прав не зависят от HTTP-процесса. Сетевые проверки должны провалиться; это подтверждено отдельным прогоном.

## Протоколы проверок

### bash_basics_healthy

Полный протокол: evidence/bash_basics_healthy.md.

```
2026-11-08 19:50:46 PASS: Заголовок CSV
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Каждая строка содержит 3 поля
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: HTTP доступен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: HTTP-код 200
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: HTTP содержит Ivan
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Успешный ответ без error
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: TCP-порт открыт
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Ответ быстрее 1 секунды
```

```
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:46 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0
```

### bash_real_stop

Полный протокол: evidence/bash_real_stop.md.

```
2026-11-08 19:50:50 PASS: Заголовок CSV
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 PASS: Каждая строка содержит 3 поля
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 FAIL: HTTP доступен
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 FAIL: HTTP-код 200
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 FAIL: HTTP содержит Ivan
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 FAIL: Успешный ответ без error
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 FAIL: TCP-порт открыт
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 FAIL: Ответ быстрее 1 секунды
TOTAL=1 PASS=0 FAIL=1
2026-11-08 19:50:50 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:50 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0
```

## Протоколы проверок

### cron

Полный протокол: evidence/cron.md.

```
*/5 * * * * /LoL/.scratch/mdk-coursework/cron-job.sh bash5
*/15 * * * * /LoL/.scratch/mdk-coursework/cron-job.sh json15

2026-11-08T19:40:00+03:00 scheduled=bash5
2026-11-08T19:45:00+03:00 scheduled=json15
```

```
2026-11-08T19:45:00+03:00 scheduled=bash5
2026-11-08T19:50:00+03:00 scheduled=bash5
```

## bash-scheduled.log

```
2026-11-08 19:40:00 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:00 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Заголовок CSV
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каждая строка содержит 3 поля
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP доступен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP-код 200
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP содержит Ivan
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Успешный ответ без error
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: TCP-порт открыт
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Ответ быстрее 1 секунды
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0
```

## kernel-scheduled.log

```
2026-11-08 19:50:00 PASS: Заголовок CSV
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каждая строка содержит 3 поля
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Файл существует и не пуст
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP доступен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP-код 200
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: HTTP содержит Ivan
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каталог логов доступен на запись
```

```
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: Успешный ответ без error
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:00 PASS: TCP-порт открыт
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:01 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:01 PASS: Ответ быстрее 1 секунды
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:01 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:50:01 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0
```

## json-scheduled.log

```
2026-11-08 19:45:00 PASS: JSON email_format
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:00 PASS: UTF-8 broken.json
2026-11-08 19:45:00 PASS: UTF-8 config.json
2026-11-08 19:45:00 PASS: UTF-8 orders.json
2026-11-08 19:45:00 PASS: UTF-8 products.json
2026-11-08 19:45:00 PASS: UTF-8 users.json
TOTAL=5 PASS=5 FAIL=0
2026-11-08 19:45:00 PASS: users.json существует
2026-11-08 19:45:00 PASS: config.json существует
TOTAL=2 PASS=2 FAIL=0
2026-11-08 19:45:00 PASS: JSON fields
TOTAL=1 PASS=1 FAIL=0
SKIP: jq отсутствует; необязательная зависимость
2026-11-08 19:45:00 PASS: JSON no_extra_fields
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:00 PASS: JSON orders
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:00 PASS: JSON products
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON ranges
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON schema
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON types
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON unique_ids
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON users
TOTAL=1 PASS=1 FAIL=0
2026-11-08 19:45:01 PASS: JSON valid
TOTAL=1 PASS=1 FAIL=0
```

## Вывод

Положительный и отрицательный прогоны разделены. Намеренный дефект обнаружен, исходные данные не повреждены. Полные протоколы и способ повторения входят в репозиторий; ограничения стенда явно указаны.

## Использованные материалы

Условия работ: репозитории softboxdev/programming_course и softboxdev/test_methodologies; точные ссылки и commit SHA зафиксированы в SOURCES.md.

Документация: Python sqlite3, Python json, Flask Quickstart, Postman Newman.

# Приложение. Листинги файлов

## testing/bash_basics/check.sh

```
#!/usr/bin/env bash
set -u
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
DATA="${QA_DATA:-$ROOT/data}"
RUN="${QA_RUN:-$ROOT/.logs}"
mkdir -p "$RUN"
LOG_FILE="$RUN/test_$(date +%Y%m%d_%H%M%S)_$$.log"
PASS=0
FAIL=0
check() {
    local name="$1"
    shift
    if "$@" >"$RUN/last_stdout.txt" 2>"$RUN/last_stderr.txt"; then
        echo "$(date '+%F %T') PASS: $name" | tee -a "$LOG_FILE"
        PASS=$((PASS + 1))
    else
        echo "$(date '+%F %T') FAIL: $name" | tee -a "$LOG_FILE" "$RUN/alerts.log"
        cat "$RUN/last_stderr.txt" >> "$LOG_FILE"
        FAIL=$((FAIL + 1))
    fi
}
finish() {
    echo "TOTAL=$((PASS + FAIL)) PASS=$PASS FAIL=$FAIL" | tee -a "$LOG_FILE"
    test "$FAIL" -eq 0
}
```

## testing/bash_basics/data/users.csv

```
id,name,status
1,Ivan,active
2,Maria,inactive
```

## testing/bash_basics/functions.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/check.sh"
URL="${QA_URL:-http://127.0.0.1:8080/users.csv}"
HOST="${QA_HOST:-127.0.0.1}"
PORT="${QA_PORT:-8080}"
fetch() { curl -fsS --max-time 3 "$URL"; }
file_rows() { test "$(wc -l < "$DATA/users.csv")" -eq 3; }
file_fields() { awk -F, 'NF!=3{bad=1} END{exit bad}' "$DATA/users.csv"; }
unique_ids() { test -z "$(tail -n +2 "$DATA/users.csv" | cut -d, -f1 | sort | uniq -d)"; }
http_code() { test "$(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")" = 200; }
http_content() { local body; body="$(fetch)" || return 1; [[ "$body" == *Ivan* ]]; }
no_errors() { local body; body="$(fetch)" || return 1; ! printf '%s' "$body" | grep -qi error; }
process_alive() { test -n "${QA_PID:-}" && kill -0 "$QA_PID" 2>/dev/null; }
```

```
port_open() { timeout 2 bash -c 'exec 3<>/dev/tcp/$1/$2' bash "$HOST" "$PORT"; }
response_fast() { local seconds; seconds="$(curl -fsS -o /dev/null -w '%{time_total}' --max-time 3
"$URL")" || return 1; awk -v t="$seconds" 'BEGIN{exit !(t<1)}'; }
```

## testing/bash_basics/run_all.sh

```
#!/usr/bin/env bash
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
result=0
for script in "$ROOT"/test_*.sh; do
    bash "$script" || result=1
done
exit "$result"
```

## testing/bash_basics/run_tests.sh

```
#!/usr/bin/env bash
exec bash "$(dirname "${BASH_SOURCE[0]}")/run_all.sh" "$@"
```

## testing/bash_basics/test_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Заголовок CSV" grep -q "^id,name,status$" "$DATA/users.csv"
finish
```

## testing/bash_basics/test_csv_format.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каждая строка содержит 3 поля" file_fields
finish
```

## testing/bash_basics/test_file.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл существует и не пуст" test -s "$DATA/users.csv"
finish
```

### testing/bash_basics/test_http.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP доступен" fetch
finish
```

### testing/bash_basics/test_http_code.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP-код 200" http_code
finish
```

### testing/bash_basics/test_http_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP содержит Ivan" http_content
finish
```

### testing/bash_basics/test_lines.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "В файле ровно 3 строки" file_rows
finish
```

### testing/bash_basics/test_log_writable.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов доступен на запись" test -w "$RUN"
finish
```

### testing/bash_basics/test_logs.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов существует" test -d "$RUN"
finish
```

### testing/bash_basics/test_no_errors.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Успешный ответ без error" no_errors
finish
```

### testing/bash_basics/test_port.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "TCP-порт открыт" port_open
finish
```

### testing/bash_basics/test_process.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Собственный процесс сервера запущен" process_alive
finish
```

### testing/bash_basics/test_response_time.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Ответ быстрее 1 секунды" response_fast
finish
```

### testing/bash_basics/test_rights.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл читается" test -r "$DATA/users.csv"
finish
```

### testing/bash_basics/test_unique_ids.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "ID уникальны" unique_ids
finish
```